

Nama : Mahdi Sarwan Abdullah

Nim : 2509106067

Kelas : Infor B25

1. Command line "mkdir ./posttest4/" untuk membuat direktori "posttest4" di dalam direktori saat ini. Command line "ls -l" untuk menampilkan isi current dir yang kita tempati dengan format list vertikal. Terdapat informasi dir "posttest4" yang berarti direktori berhasil dibuat. Command line "cd ./posttest4/" untuk mencari direktori posttest4 pada direktori saat ini dan masuk ke dalam dir tersebut. Baris path terakhir menunjukkan kita berhasil berpindah ke dalam dir posttest4. Command line "micro tiket_konser.py" untuk membuat file baru bernama "tiket_konser.py" dan sekaligus masuk ke dalam file tersebut menggunakan text editor micro. Pada gambar terakhir, terdapat baris command yang kosong dan "tiket_konser.py" di paling bawah yang berarti langkah 1 berhasil.

```
mahdi-067@praktikum-ubuntu:~$ mkdir ./posttest4/
mahdi-067@praktikum-ubuntu:~$ ls -l
total 8
drwxrwxr-x 2 mahdi-067 mahdi-067 4096 Oct  3 11:21 posttest4
drwx----- 3 mahdi-067 mahdi-067 4096 Oct  1 06:09 snap
mahdi-067@praktikum-ubuntu:~$ cd ./posttest4/
mahdi-067@praktikum-ubuntu:~/posttest4$ micro tiket_konser.py
```



2. Isi file tiket_konser.py. Import threading untuk menggunakan library dan method yang tersedia pada thread. Import time untuk menggunakan library dan method yang tersedia pada time. Tambahkan

variabel global sisa tiket dan berikan nilai 100. Buat fungsi "beli_tiket" untuk logika pembelian tiketnya. Command line "global sisa_tiket" untuk mengakses dan memperbarui nilai pada variabel global sisa_tiket. Command line ke-8 untuk membaca nilai sisa_tiket. Command line ke-9 untuk melakukan penundaan micro selama 0.001 detik. Command line ke-10 untuk mengurangi sisa_tiket sebanyak 1. Buat list dengan nama threads. Jalankan 100 threads secara bersamaan dengan looping for seperti gambar di bawah. Looping for kedua untuk memastikan semua thread telah selesai dijalankan. Save dan keluar dari file. Command line "ls -l" untuk menampilkan isi dir saat ini dengan format list vertikal. Pada gambar kedua, terdapat informasi file "tiket_konser.py" yang berarti langkah 2 berhasil.

```
1 import threading
2 import time
3
4 sisa_tiket = 100
5
6 def beli_tiket(id_pembeli):
7     global sisa_tiket
8     nilai_sekarang = sisa_tiket
9     time.sleep(0.001)
10    sisa_tiket = nilai_sekarang - 1
11
12 threads = []
13
14 for i in range(100):
15     t = threading.Thread(target=beli_tiket, args=(i,))
16     threads.append(t)
17     t.start()
18
19 for t in threads:
20     t.join()
21
22 print(sisa_tiket)_
```

```
mahdi-067@praktikum-ubuntu:~/posttest4$ ls -l
total 4
-rw-rw-r-- 1 mahdi-067 mahdi-067 374 Oct  3 12:00 tiket_konser.py
mahdi-067@praktikum-ubuntu:~/posttest4$
```

3. Command line "python3 tiket_konser.py" untuk mengeksekusi file tiket_konser.py dengan interpreter python. Lakukan perintah tersebut sebanyak 3 kali. Pada stdout pertama, terdapat angka 98. Pada stdout kedua dan ketiga, terdapat angka 99. Berarti program tiket_konser.py terjadi anomali/race condition dengan menampilkan sisa_tiket akhir bernilai selain 0. Dengan begini, langkah 3 berhasil.

```
mahdi-067@praktikum-ubuntu:~/posttest4$ python3 tiket_konser.py
98
mahdi-067@praktikum-ubuntu:~/posttest4$ python3 tiket_konser.py
99
mahdi-067@praktikum-ubuntu:~/posttest4$ python3 tiket_konser.py
99
mahdi-067@praktikum-ubuntu:~/posttest4$
```

4. Command line “micro tiket_konser.py” untuk mengedit file tiket_konser.py kembali dengan text editor micro. Ubah file dengan menambahkan logika mutex (mutual exclusion) dimana akan diterapkan pada variabel "kunci". Command line ke-4 untuk membuat kunci thread menggunakan variabel "kunci". Command line ke-9 adalah penerapan mutex secara implisit, dimana logikanya kunci akan membiarkan thread masuk (locking yang berarti tidak boleh ada thread yang boleh masuk) dan setelah blok indentasi selesai, kunci thread akan melepaskan thread (thread lain sudah boleh masuk kembali). Save dan keluar dari file. Dengan begini, langkah 4 berhasil.

```
mahdi-067@praktikum-ubuntu:~/posttest4$ micro tiket_konser.py
```

```
1 import threading
2 import time
3
4 kunci = threading.Lock()
5 sisa_tiket = 100
6
7 def beli_tiket(id_pembeli):
8     global sisa_tiket
9     with kunci:
10         nilai_sekarang = sisa_tiket
11         time.sleep(0.001)
12         sisa_tiket = nilai_sekarang - 1
13
14 threads = []
15
16 for i in range(100):
17     t = threading.Thread(target=beli_tiket, args=(i,))
18     threads.append(t)
19     t.start()
20
21 for t in threads:
22     t.join()
23
24 print(sisa_tiket)
```

5. Command line “python3 tiket_konser.py” untuk mengeksekusi file tiket_konser.py dengan interpreter python. Lakukan perintah tersebut berkali-kali. Pada setiap stdout, terdapat angka 0. Berarti program tiket_konser.py sudah berjalan normal tanpa anomali dengan ditandai program selalu menampilkan sisa_tiket akhir konsisten bernilai 0. Dengan begini, langkah 5 berhasil.

```
mahdi-067@praktikum-ubuntu:~/posttest4$ python3 tiket_konser.py
0
mahdi-067@praktikum-ubuntu:~/posttest4$ python3 tiket_konser.py
0
mahdi-067@praktikum-ubuntu:~/posttest4$ python3 tiket_konser.py
0
mahdi-067@praktikum-ubuntu:~/posttest4$ _
```

6. Program:

```
1 import threading
2 import time
3
4 kunci = threading.Lock()
5 sisa_tiket = 100
6
7 def beli_tiket(id_pembeli):
8     global sisa_tiket
9     with kunci:
10         nilai_sekarang = sisa_tiket
11         time.sleep(0.001)
12         sisa_tiket = nilai_sekarang - 1
13
14 threads = []
15
16 for i in range(100):
17     t = threading.Thread(target=beli_tiket, args=(i,))
18     threads.append(t)
19     t.start()
20
21 for t in threads:
22     t.join()
23
24 print(sisa_tiket)
```

Output:

```
mahdi-067@praktikum-ubuntu:~/posttest4$ python3 tiket_konser.py
0
mahdi-067@praktikum-ubuntu:~/posttest4$ python3 tiket_konser.py
0
mahdi-067@praktikum-ubuntu:~/posttest4$ python3 tiket_konser.py
0
mahdi-067@praktikum-ubuntu:~/posttest4$ _
```

a. Teori Umum:

Mutex bekerja seperti kunci biner (0 atau 1). Hanya 1 thread yang diizinkan memegang kunci pada satu waktu. Thread lain harus menunggu thread tersebut selesai dan melepaskan kunci yang akan diambil oleh thread selanjutnya. Pernyataan “with” menggunakan protokol `__enter__()` dan `__exit__()`. Saat masuk blok with, metode `__enter__()` secara implisit memanggil `acquire()` (mengambil kunci). Saat keluar dari blok with, metode `__exit__()` secara implisit memanggil `release()` (melepaskan kunci).

b. Analisis kode:

Entry Section (baris ke-9): di sini jalan masuk logika mutex (lock) dimana ditandai secara implisit oleh pernyataan "with".

Critical Section (baris 10—12): di sini logika untuk melakukan modifikasi variabel bersama "sisa_tiket".

Exit Section (baris ke-12 saat blok indentasi sudah selesai): di sini jalan keluar logika mutex (unlock) dimana di balik layar sebenarnya thread otomatis melepaskan kunci.

Remainder Section (tidak ada): blok kode fungsi selesai pada baris ke-12. Sedangkan, setelah exit section (baris ke-12 selesai) tidak ada perintah lain. Maka program ini tidak punya remainder section.

c. Karena variabel `sisa_tiket` adalah variabel bersama tunggal. Saat kita menggunakan semaphore ($n > 1$), berarti ada banyak thread yang akan mengakses dan memodifikasi variabel `sisa_tiket` pada satu waktu. Ini malah akan kembali menyebabkan program mengalami race condition. Lalu, pendekatan implisit lebih aman karena saat terjadi error, maka program akan menyelesaikan logika with nya dulu (melepaskan kunci) sama seperti logika blok try dan finally. Sedangkan untuk pendekatan eksplisit, saat terjadi error sebelum kunci di release, maka program akan mengalami deadlock. Selain itu, pernyataan with lebih efisien dan ringkas (clean code) dari pada harus menuliskan blok kode try dan finally secara manual dan repetitif.